University of Central Missouri

Department of Computer Science & Cybersecurity

CS5760 Natural Language Processing

Spring 2026

Homework 4.

Student name: Janani Govuri

700773302

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Submit your GitHub link on the Bright Space.
- Comment your code appropriately ***IMPORTANT.***
- Any submission after provided deadline is considered as a late submission.

Q6. Multi-Input Feedforward

Consider a neural network with 3 inputs x1=2, x2=1, x3=3, one hidden layer of 2 sigmoid units, and one sigmoid output.
Weights:

$$A = \begin{bmatrix} 0.2 & -0.5 \\ 0.1 & 0.3 \\ -0.2 & 0.8 \\ 0.4 & -0.6 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7 \\ -1.2 \\ 0.5 \end{bmatrix}$$

- A includes biases (last row).
- B includes bias (last element).

Tasks
a) Compute hidden pre-activations and activations.

b) Compute the output activation y.

c) If the true label is t=1, calculate the binary cross-entropy loss.

Q7. XOR with ReLU Network

We use the XOR ReLU network from the slides

$$h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1), \quad y = \mathrm{ReLU}(h_1 - 2h_2)$$

Extend it with an additional hidden unit:

$$h_3 = \mathrm{ReLU}(2x_1 - x_2)$$

and new output:

$$y = \mathrm{ReLU}(h_1 - 2h_2 + h_3)$$

Tasks
a) Compute outputs for all four XOR inputs (0,0),(0,1),(1,0),(1,1).

b) Compare the decision boundary with the original 2-hidden-unit XOR network.

c) Does this extension still compute XOR exactly? If not, which inputs differ?

Q8. Decision Boundary and Misclassification

You train a perceptron with decision rule:

$$y = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

with parameters w1=1, w2=−2, b=1.

Dataset:
(2,1)→1, (1,3)→0, (3,2)→1, (0,1)→0.

Tasks
a) Write the decision boundary equation and sketch it.

on the first mistake, what are the new weights?

Part I. Short Answer

Q3. Neural Networks: Structure, Activation, and Representation

(a) Why are non-linear activation functions (like ReLU, sigmoid, tanh) necessary in neural networks?

Non-linear activation functions (ReLU, sigmoid, tanh) allow neural networks to learn **complex and non-linear relationships**. Without them, the network behaves like a **linear model**, even with multiple layers, and cannot solve problems like XOR.

b) Why are deep networks better at feature learning compared to shallow models like logistic regression?

Deep networks learn **hierarchical features** (low-level → high-level patterns), while shallow models like logistic regression capture only **simple linear relationships**.

1) RNN Families & Use-Cases (Many-to-X)
a) Map each task to the most suitable RNN I/O pattern and explain why (1–2 lines each):
• next-word prediction → many-to-many (aligned)

Each input word produces an output (next word) at every time step, so input and output sequences are aligned.

• sentiment of a sentence → many-to-one

The entire sequence of words is processed to produce a single output (overall sentiment).

• NER → many-to-many (aligned)

Each input word is mapped to a corresponding label (entity tag), so outputs align with inputs.

• machine translation → many-to-many (unaligned)

Input and output sequences differ in length and alignment, as the translated sentence structure may change.
(Choose from: one-to-many, many-to-one, many-to-many aligned, many-to-many unaligned.)
b) In one sentence, explain how "unrolling" over time enables BPTT and weight sharing.

Unrolling converts the RNN into a sequence of repeated units across time, enabling **Backpropagation Through Time (BPTT)** and **weight sharing**.
c) Give one advantage and one limitation of weight sharing across time in RNNs.

Advantage: reduces parameters and improves generalization.

Limitation: limits flexibility for time-specific patterns.

2) Vanishing Gradients & Remedies
a) Describe the vanishing gradient problem in RNNs and how it affects long-range dependencies.

In RNNs, during backpropagation through many time steps, gradients get multiplied repeatedly and become very small (close to zero). As a result, the network cannot effectively learn from earlier time steps, making it difficult to capture **long-range dependencies**.

b) List two architectural solutions and briefly how each helps gradient flow.

LSTM (Long Short-Term Memory):

Uses gates and a cell state that allow information and gradients to flow more easily over long sequences, reducing vanishing.

GRU (Gated Recurrent Unit):

Uses update and reset gates to control information flow, helping preserve important gradients with a simpler structure than LSTM.

c) Give one *training* technique (not an architecture change) that can mitigate the issue and why.

Gradient clipping:

Limits the magnitude of gradients during training, preventing them from becoming too small or too large, which stabilizes learning and improves performance.

3) LSTM Gates & Cell State

a) Explain the roles of the forget, input, and output gates. For each, name its activation and purpose.

Forget gate (sigmoid): Decides which information from the previous cell state should be removed.

Input gate (sigmoid): Controls which new information should be added to the cell state.

Output gate (sigmoid): Determines which part of the cell state is exposed as the hidden state (output).

b) Why is the LSTM cell state often descri

The LSTM cell state allows information to flow with minimal changes across time steps, creating a nearly linear path that helps gradients pass without vanishing.

c) In one or two sentences, contrast "what to remember" vs. "what to expose" in LSTMs.

"Remember" refers to storing important information in the cell state, while "expose" refers to selecting relevant parts of that information to output through the hidden state.

4) Self-Attention

a) Define Query (Q), Key (K), and Value (V) in the context of self-attention.

Query (Q): Represents the current word/token that is trying to gather information.

Key (K): Represents all tokens in the sequence used for matching relevance with the query.

Value (V): Contains the actual information of each token that will be weighted and combined.

b) Write the formula for dot-product attention.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

?

Dividing *dk\sqrt{d_k}*dk prevents the dot-product values from becoming too large, which stabilizes the softmax function and ensures better gradient flow during training.

5) Multi-Head Attention & Residual Connections

a) Why do Transformers use multi-head attention instead of single-head attention?

Transformers use multi-head attention to capture **different types of relationships simultaneously** (e.g., syntax, semantics) by attending to information from multiple representation subspaces.

LayerNorm)? Explain two benefits.

Residual connection (Add): helps preserve original input and improves gradient flow.
Layer Normalization (Norm): stabilizes training and speeds up convergence.

c) Describe one example of linguistic
that different heads might capture (e.g., coreference, syntax).

One attention head may capture **subject–verb agreement (syntax)**, while another captures **coreference (e.g., linking "he" to a person mentioned earlier)**.

6) Encoder–Decoder with Masked Attention

a) Why does the decoder use masked self-attention? What problem does it prevent?

Masked self-attention prevents the decoder from accessing **future tokens**, ensuring that predictions depend only on previously generated tokens and preserving the correct sequence order.

Encoder self-attention: each input token attends to all other tokens in the input sequence.

Encoder–decoder cross-attention: decoder tokens attend to encoder outputs to gather relevant information from the input.

c) During inference (no teacher forcing), how does the model generate tokens step by step?

The model generates tokens **one at a time**, using previously generated tokens as input, and continues this process until an end-of-sequence token is produced.

Part II: Programming

Q1. Character-Level RNN Language Model ("hello" toy & beyond)
Goal: Train a tiny character-level RNN to predict the next character given previous characters.

Data (toy to start):

- Start with a small toy corpus you create (e.g., several "hello…", "help…", short words/sentences).
- Then expand to a short plain-text file of ~50–200 KB (any public-domain text of your choice).

Model:

- Embedding → RNN (Vanilla RNN or GRU or LSTM) → Linear → Softmax over characters.
- Hidden size 64–256; sequence length 50–100; batch size 64; train 5–20 epochs.

Train:

- Teacher forcing (use the true previous char as input during training).
- Cross-entropy loss; Adam optimizer.

Report:

1. Training/validation loss curves.
2. Sample 3 temperature-controlled generations (e.g., $\tau = 0.7, 1.0, 1.2$) for 200–400 chars each.
3. A 3–5 sentence reflection: what changes when you vary sequence length, hidden size, and temperature?
   (Connect to slides: embedding, sampling loop, teacher forcing, tradeoffs

Code:

Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1]

```python
import torch
import torch.nn as nn
import torch.optim as optim
import random

# ---- Sample Toy Data ----
text = "hello help hello hero hello hill hello help hello"
chars = sorted(list(set(text)))

# Create mappings
char2idx = {ch: i for i, ch in enumerate(chars)}
idx2char = {i: ch for ch, i in char2idx.items()}

vocab_size = len(chars)

# Encode text
data = [char2idx[ch] for ch in text]

# ---- Hyperparameters ----
hidden_size = 128
seq_length = 20
learning_rate = 0.003
epochs = 10

# ---- Model Definition ----
class CharRNN(nn.Module):
    def __init__(self, vocab_size, hidden_size):
        super(CharRNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_size)
        self.rnn = nn.LSTM(hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden):
        x = self.embedding(x)
        out, hidden = self.rnn(x, hidden)
        out = self.fc(out)
        return out, hidden
```

Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] 12s

```python
        out = self.fc(out)
        return out, hidden

# Initialize model
model = CharRNN(vocab_size, hidden_size)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# ---- Training ----
for epoch in range(epochs):
    total_loss = 0
    hidden = None

    for i in range(0, len(data) - seq_length):
        seq_in = data[i:i+seq_length]
        seq_out = data[i+1:i+seq_length+1]

        inputs = torch.tensor(seq_in).unsqueeze(0)
        targets = torch.tensor(seq_out)

        optimizer.zero_grad()

        outputs, hidden = model(inputs, hidden)
        hidden = (hidden[0].detach(), hidden[1].detach())

        loss = criterion(outputs.squeeze(), targets)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")

# ---- Text Generation ----
def generate_text(model, start_str, length=100, temperature=1.0):
    model.eval()
    input_seq = torch.tensor([char2idx[ch] for ch in start_str]).unsqueeze(0)
    hidden = None

    result = start_str
```

Untitled1.ipynb

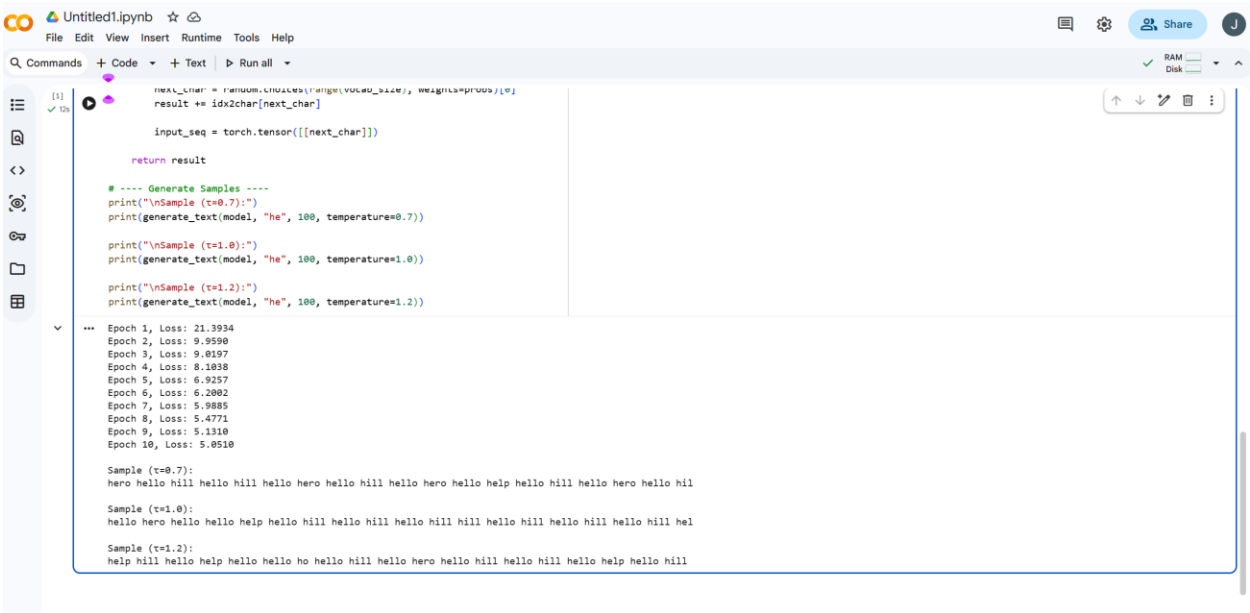
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] 12s

```python
    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")

# ---- Text Generation ----
def generate_text(model, start_str, length=100, temperature=1.0):
    model.eval()
    input_seq = torch.tensor([char2idx[ch] for ch in start_str]).unsqueeze(0)
    hidden = None

    result = start_str

    for _ in range(length):
        output, hidden = model(input_seq, hidden)
        logits = output[:, -1, :] / temperature
        probs = torch.softmax(logits, dim=1).detach().numpy()[0]

        next_char = random.choices(range(vocab_size), weights=probs)[0]
        result += idx2char[next_char]

        input_seq = torch.tensor([[next_char]])

    return result

# ---- Generate Samples ----
print("\nSample (τ=0.7):")
print(generate_text(model, "he", 100, temperature=0.7))

print("\nSample (τ=1.0):")
print(generate_text(model, "he", 100, temperature=1.0))

print("\nSample (τ=1.2):")
print(generate_text(model, "he", 100, temperature=1.2))
```

```
Epoch 1, Loss: 21.3934
Epoch 2, Loss: 9.9590
Epoch 3, Loss: 9.0197
Epoch 4, Loss: 8.1038
Epoch 5, Loss: 6.9257
Epoch 6, Loss: 6.2002
Epoch 7, Loss: 5.9885
Epoch 8, Loss: 5.4771
Epoch 9, Loss: 5.1310
```



```
Untitled1.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

        result += idx2char[next_char]

        input_seq = torch.tensor([[next_char]])

    return result

# ---- Generate Samples ----
print("\nSample (τ=0.7):")
print(generate_text(model, "he", 100, temperature=0.7))

print("\nSample (τ=1.0):")
print(generate_text(model, "he", 100, temperature=1.0))

print("\nSample (τ=1.2):")
print(generate_text(model, "he", 100, temperature=1.2))

Epoch 1, Loss: 21.3934
Epoch 2, Loss: 9.9590
Epoch 3, Loss: 9.0197
Epoch 4, Loss: 8.1038
Epoch 5, Loss: 6.9257
Epoch 6, Loss: 6.2002
Epoch 7, Loss: 5.9885
Epoch 8, Loss: 5.4771
Epoch 9, Loss: 5.1310
Epoch 10, Loss: 5.0510

Sample (τ=0.7):
hero hello hill hello hill hello hero hello hill hello hero hello help hello hill hello hero hello hil

Sample (τ=1.0):
hello hero hello hello help hello hill hello hill hello hill hill hello hill hello hill hello hill hel

Sample (τ=1.2):
help hill hello help hello hello ho hello hill hello hero hello hill hello hill hello help hello hill
```

Reflection:

Increasing sequence length improves the model's ability to capture long-term dependencies. A larger hidden size increases model capacity but may lead to overfitting. Temperature controls randomness in output: lower values produce stable text, while higher values generate more diverse but less coherent sequences. Teacher forcing helps training but may cause mismatch during inference. Overall, performance improves with better tuning and larger datasets

Q2. Mini Transformer Encoder for Sentences

Task: Build a mini Transformer Encoder (NOT full decoder) to process a batch of sentences.

Steps:

1. Use a small dataset (e.g., 10 short sentences of your choice).
2. Tokenize and embed the text.
3. Add sinusoidal positional encoding.
4. Implement:
    - Self-attention layer
    - Multi-head attention (2 or 4 heads)
    - Feed-forward layer
    - Add & Norm
5. Show:
    - Input tokens
    - Final contextual embeddings
    - Attention heatmap between words (visual or printed)

Code:

HW4Code2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import matplotlib.pyplot as plt

# -------------------------------
# 1. Sample Dataset (10 sentences)
# -------------------------------
sentences = [
    "i love nlp",
    "transformers are powerful",
    "deep learning is fun",
    "attention is important",
    "nlp is interesting",
    "i enjoy learning",
    "this is a model",
    "we love ai",
    "models learn patterns",
    "data is important"
]

# -------------------------------
# 2. Tokenization
# -------------------------------
words = list(set(" ".join(sentences).split()))
word2idx = {w: i for i, w in enumerate(words)}
idx2word = {i: w for w, i in word2idx.items()}

def tokenize(sentence):
    return [word2idx[w] for w in sentence.split()]

tokenized_data = [tokenize(s) for s in sentences]

# Padding
max_len = max(len(seq) for seq in tokenized_data)

def pad(seq):
    return seq + [0]*(max_len - len(seq))
```

HW4Code2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

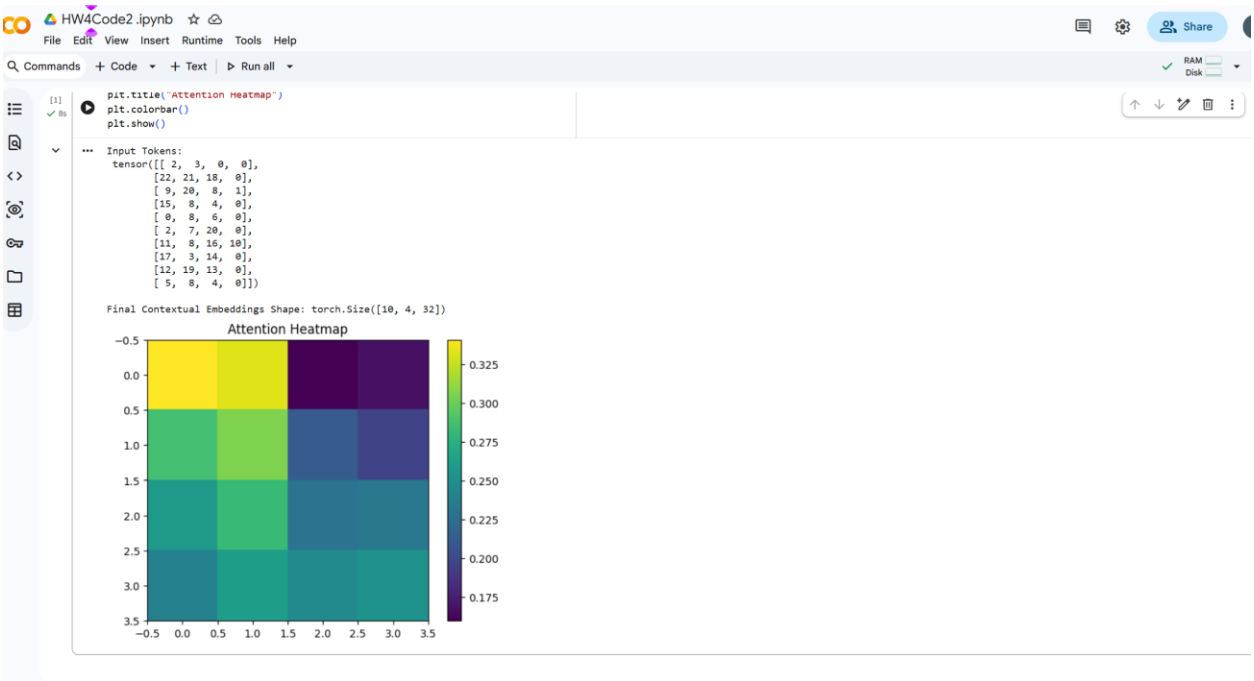
```python
max_len = max(len(seq) for seq in tokenized_data)

def pad(seq):
    return seq + [0]*(max_len - len(seq))

inputs = torch.tensor([pad(seq) for seq in tokenized_data])

print("Input Tokens:\n", inputs)

# -------------------------------
# 3. Embedding Layer
# -------------------------------
d_model = 32
embedding = nn.Embedding(len(words), d_model)
x = embedding(inputs)

# -------------------------------
# 4. Positional Encoding
# -------------------------------
def positional_encoding(max_len, d_model):
    pe = torch.zeros(max_len, d_model)
    for pos in range(max_len):
        for i in range(0, d_model, 2):
            pe[pos, i] = math.sin(pos / (10000 ** ((2*i)/d_model)))
            pe[pos, i+1] = math.cos(pos / (10000 ** ((2*i)/d_model)))
    return pe

pe = positional_encoding(max_len, d_model)
x = x + pe

# -------------------------------
# 5. Multi-Head Attention
# -------------------------------
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, heads=2):
        super().__init__()
        self.heads = heads
        self.d_k = d_model // heads

        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
```

```python
        self.d_k = d_model // heads

        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, T, C = x.size()

        Q = self.q_linear(x).view(B, T, self.heads, self.d_k).transpose(1,2)
        K = self.k_linear(x).view(B, T, self.heads, self.d_k).transpose(1,2)
        V = self.v_linear(x).view(B, T, self.heads, self.d_k).transpose(1,2)

        scores = torch.matmul(Q, K.transpose(-2,-1)) / math.sqrt(self.d_k)
        attn = torch.softmax(scores, dim=-1)

        out = torch.matmul(attn, V)
        out = out.transpose(1,2).contiguous().view(B, T, C)
        return self.out(out), attn

# ------------------------------
# 6. Feed Forward Layer
# ------------------------------
class FeedForward(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.fc1 = nn.Linear(d_model, 64)
        self.fc2 = nn.Linear(64, d_model)

    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))

# ------------------------------
# 7. Transformer Encoder Block
# ------------------------------
class EncoderBlock(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.attn = MultiHeadAttention(d_model)
        self.norm1 = nn.LayerNorm(d_model)
```

Commands + Code + Text Run all

```python
    def __init__(self, d_model):
        super().__init__()
        self.attn = MultiHeadAttention(d_model)
        self.norm1 = nn.LayerNorm(d_model)
        self.ff = FeedForward(d_model)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, x):
        attn_out, attn_weights = self.attn(x)
        x = self.norm1(x + attn_out)

        ff_out = self.ff(x)
        x = self.norm2(x + ff_out)

        return x, attn_weights

# ------------------------------
# 8. Run Encoder
# ------------------------------
encoder = EncoderBlock(d_model)
output, attn_weights = encoder(x)

print("\nFinal Contextual Embeddings Shape:", output.shape)

# ------------------------------
# 9. Attention Heatmap
# ------------------------------
# Show attention for first sentence, first head
heatmap = attn_weights[0][0].detach().numpy()

plt.imshow(heatmap, cmap='viridis')
plt.title("Attention Heatmap")
plt.colorbar()
plt.show()
```

```
Input Tokens:
 tensor([[ 2,  3,  0,  0],
        [22, 21, 18,  0],
        [ 9, 20,  8,  1],
        [15,  8,  4,  0],
        [ 0,  8,  6,  0],
```



Q3. Implement Scaled Dot-Product Attention

Goal: Implement the attention function from your slides:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Requirements:

- Write a function in PyTorch or TensorFlow to compute attention.
- Test it using random Q, K, V inputs.
- Print:
  - Attention weight matrix
  - Output vectors
  - Softmax stability check (before and after scaling)

Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3]

```python
import torch
import torch.nn.functional as F
import math

# ------------------------------
# 1. Scaled Dot-Product Attention Function
# ------------------------------
def scaled_dot_product_attention(Q, K, V):
    d_k = Q.size(-1)

    # Compute raw scores
    scores = torch.matmul(Q, K.transpose(-2, -1))

    # Without scaling (for comparison)
    unscaled_softmax = F.softmax(scores, dim=-1)

    # Apply scaling
    scaled_scores = scores / math.sqrt(d_k)

    # Softmax after scaling
    attn_weights = F.softmax(scaled_scores, dim=-1)

    # Final output
    output = torch.matmul(attn_weights, V)

    return output, attn_weights, scores, scaled_scores, unscaled_softmax

# ------------------------------
# 2. Test with Random Q, K, V
# ------------------------------
torch.manual_seed(0)

batch_size = 1
seq_len = 4
d_k = 8

Q = torch.randn(batch_size, seq_len, d_k)
K = torch.randn(batch_size, seq_len, d_k)
V = torch.randn(batch_size, seq_len, d_k)
```

[3]

```python
Q = torch.randn(batch_size, seq_len, d_k)
K = torch.randn(batch_size, seq_len, d_k)
V = torch.randn(batch_size, seq_len, d_k)

# Run attention
output, attn_weights, scores, scaled_scores, unscaled_softmax = scaled_dot_product_attention(Q, K, V)

# ------------------------------
# 3. Print Results
# ------------------------------
print("=== Raw Scores (QK^T) ===")
print(scores)

print("\n=== Softmax WITHOUT Scaling ===")
print(unscaled_softmax)

print("\n=== Scaled Scores ===")
print(scaled_scores)

print("\n=== Attention Weights (WITH Scaling) ===")
print(attn_weights)

print("\n=== Output Vectors ===")
print(output)
```

```
=== Raw Scores (QK^T) ===
tensor([[[-0.0146,  5.1091, -0.3921, -3.7779],
         [ 0.4648,  0.5446, -0.7034,  0.9571],
         [ 1.3816, -7.0338, -0.2771,  2.2690],
         [ 0.6756,  3.8463, -1.2603, -2.3062]]])

=== Softmax WITHOUT Scaling ===
tensor([[[5.8939e-03, 9.8993e-01, 4.0409e-03, 1.3678e-04],
         [2.4813e-01, 2.6875e-01, 7.7146e-02, 4.0597e-01],
         [2.7628e-01, 6.1181e-05, 5.2604e-02, 6.7106e-01],
         [3.9970e-02, 9.5224e-01, 5.7673e-03, 2.0266e-03]]])

=== Scaled Scores ===
tensor([[[-0.0052,  1.8063, -0.1386, -1.3357],
         [ 0.1643,  0.1926, -0.2487,  0.3384],
         [ 0.4885, -2.4868, -0.0980,  0.8022],
         [ 0.2389,  1.3599, -0.4456, -0.8154]]])
```

Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

RAM Disk

[3]

```python
print("\n=== Scaled Scores ===")
print(scaled_scores)

print("\n=== Attention Weights (WITH Scaling) ===")
print(attn_weights)

print("\n=== Output Vectors ===")
print(output)
```

```
=== Raw Scores (QK^T) ===
tensor([[[-0.0146,  5.1091, -0.3921, -3.7779],
         [ 0.4648,  0.5446, -0.7034,  0.9571],
         [ 1.3816, -7.0338, -0.2771,  2.2690],
         [ 0.6756,  3.8463, -1.2603, -2.3062]]])

=== Softmax WITHOUT Scaling ===
tensor([[[5.8939e-03, 9.8993e-01, 4.0409e-03, 1.3678e-04],
         [2.4813e-01, 2.6875e-01, 7.7146e-02, 4.0597e-01],
         [2.7628e-01, 6.1181e-05, 5.2604e-02, 6.7106e-01],
         [3.9970e-02, 9.5224e-01, 5.7673e-03, 2.0266e-03]]])

=== Scaled Scores ===
tensor([[[-0.0052,  1.8063, -0.1386, -1.3357],
         [ 0.1643,  0.1926, -0.2487,  0.3384],
         [ 0.4885, -2.4868, -0.0980,  0.8022],
         [ 0.2389,  1.3599, -0.4456, -0.8154]]])

=== Attention Weights (WITH Scaling) ===
tensor([[[0.1211, 0.7410, 0.1060, 0.0320],
         [0.2577, 0.2651, 0.1705, 0.3067],
         [0.3360, 0.0171, 0.1869, 0.4599],
         [0.2032, 0.6235, 0.1025, 0.0708]]])

=== Output Vectors ===
tensor([[[-1.2940, -0.1495, -0.3701, -0.2156, -0.9138,  1.0640, -0.0503,
           0.6983],
         [-0.6700,  0.9147, -0.5610,  0.1849, -0.8407,  0.5956,  0.0283,
           0.5789],
         [-0.3498,  1.4870, -0.6570,  0.4022, -0.8382,  0.3671,  0.0483,
           0.5201],
         [-1.1585,  0.0777, -0.2926, -0.0528, -0.9533,  1.1081, -0.0749,
           0.6575]]])
```